AI-Driven Real Estate Educational Bot Report

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vidhi Yadav	EN22CS3011077
2.	Suhani Pathak	EN22CS301990
3.	Surbhi Jadam	EN22CS3011002
4.	Vanshika Soni	EN22CS3011061
5.	Vinay	EN22CS3011082

Group Name: Group07D9

Project Number: GAI-07

Industry Mentor Name: Mr. Rishabh Tripathi

University Mentor Name: Prof. Vineeta Rathore

Academic Year: 2026

Table of Contents

1. **Problem Statement & Objectives**

   1.1 Problem Statement

   1.2 Project Objectives

   1.3 Scope of the Project

2. **Proposed Solution**

   2.1 Key features

   2.2 Overall Architecture / Workflow

   2.3 Tools & Technologies Used

3. **Results & Output**

   3.1 Screenshots / outputs

   3.2 Reports / dashboards / models

   3.3 Key outcomes

4. **Conclusion**

5. **Future Scope & Enhancements**

1. Problem Statement & Objectives

1.1 Problem Statement

Understanding real estate concepts such as property valuation, legal disclosures, and closing costs can be difficult for beginners. Traditional learning methods like reading long documents or watching lectures often do not provide interactive learning experiences.

There is a need for an intelligent system that can simplify complex real estate topics and help users learn through interactive quizzes and explanations.

PropTrain AI solves this problem by using **Artificial Intelligence and Retrieval-Augmented Generation (RAG)** to create quiz questions based on real estate knowledge and provide clear explanations to help users learn effectively.

1.2 Project Objectives

The main objectives of this project are:

1. To develop an AI-powered educational bot for learning real estate concepts.
2. To generate dynamic MCQ questions using a Large Language Model.
3. To evaluate user answers and provide detailed explanations.
4. To use RAG architecture so that the AI answers are based on real knowledge instead of hallucinated information.
5. To provide an interactive learning interface using Streamlit.

1.3 Scope of the Project

The scope of the PropTrain AI system includes:

- Generating MCQ questions from a curated real estate knowledge base.
- Providing explanations and examples for each answer.
- Tracking the user's quiz performance during the session.
- Running locally as a single-user application.

The system currently focuses on **real estate education topics** such as:

- MLS terminology
- Property valuation
- Cap rates
- Legal disclosures
- Closing costs

The project does not include:

- Multi-user support
- Cloud deployment
- Long-term data storage

2. Proposed Solution

The proposed solution is an **AI-Driven Quiz Bot** that uses **Retrieval-Augmented Generation (RAG)** with the **Google Gemini 2.5 Flash Large Language Model**.

The system retrieves relevant knowledge from a real estate knowledge base using **FAISS** vector similarity search and sends that context to Gemini to generate quiz questions and explanations.

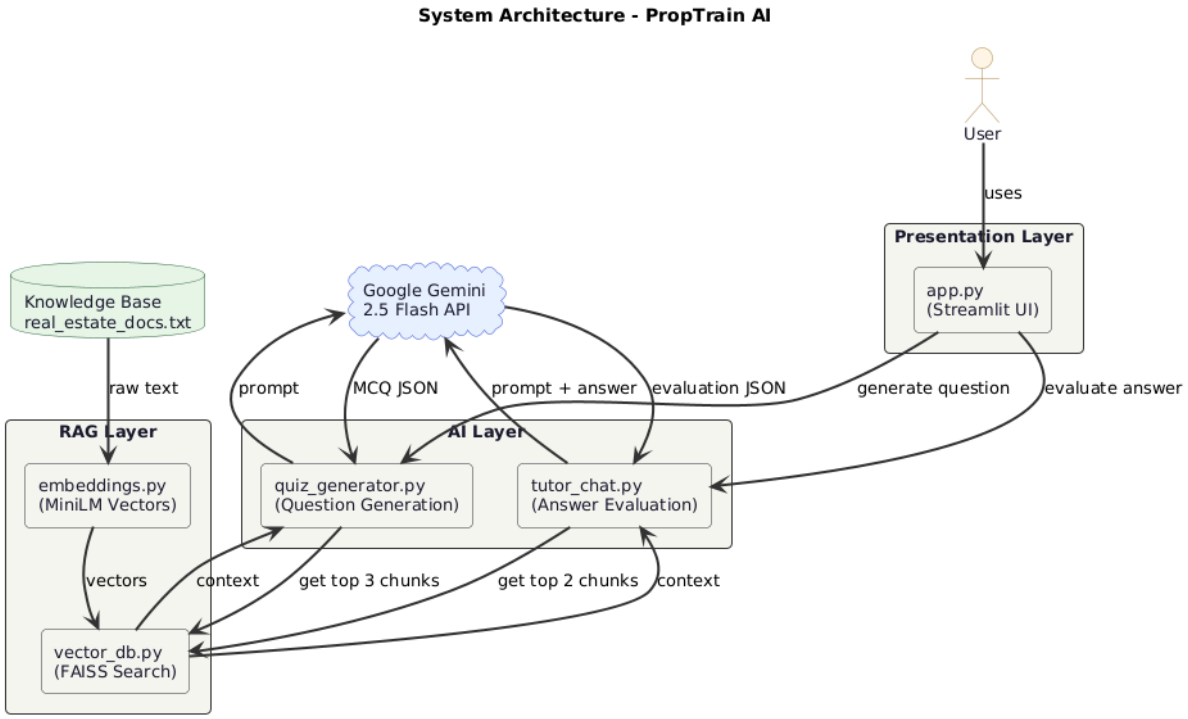
This approach ensures that all generated content is grounded in real educational material rather than relying only on the AI model.

2.1 Key Features

The main features of the PropTrain AI system include:

- AI-generated multiple-choice questions.
- Real-time answer evaluation.
- Plain-English explanations for learning.
- Real-world examples to improve understanding.
- Memory tips to help users remember concepts.
- Session tracking showing score and accuracy.
- Beginner-friendly interface using Streamlit.
- Retrieval-based AI responses using FAISS.

2.2 Overall Architecture / Workflow



The system architecture consists of four major layers.

1. Presentation Layer

This layer is implemented using **Streamlit** and provides the user interface.
It displays the quiz questions, answer options, explanations, and session statistics.

Main file: *app.py*

Responsibilities:

- Display quiz UI
- Manage session state
- Handle user interactions

2. AI Layer

This layer communicates with the **Google Gemini 2.5 Flash API**.

Modules:

- *quiz_generator.py* → Generates MCQ questions
- *tutor_chat.py* → Evaluates answers and provides explanations

3. RAG Layer

This layer retrieves relevant knowledge from the knowledge base.

Modules:

- *embeddings.py* → Creates vector embeddings
- *vector_db.py* → Stores and retrieves vectors using FAISS

Steps:

1. Convert text into embeddings.
2. Store embeddings in FAISS.
3. Retrieve relevant context during question generation.

4. Knowledge Base

The system uses a curated text file:

File:
real_estate_docs.txt

Contents:

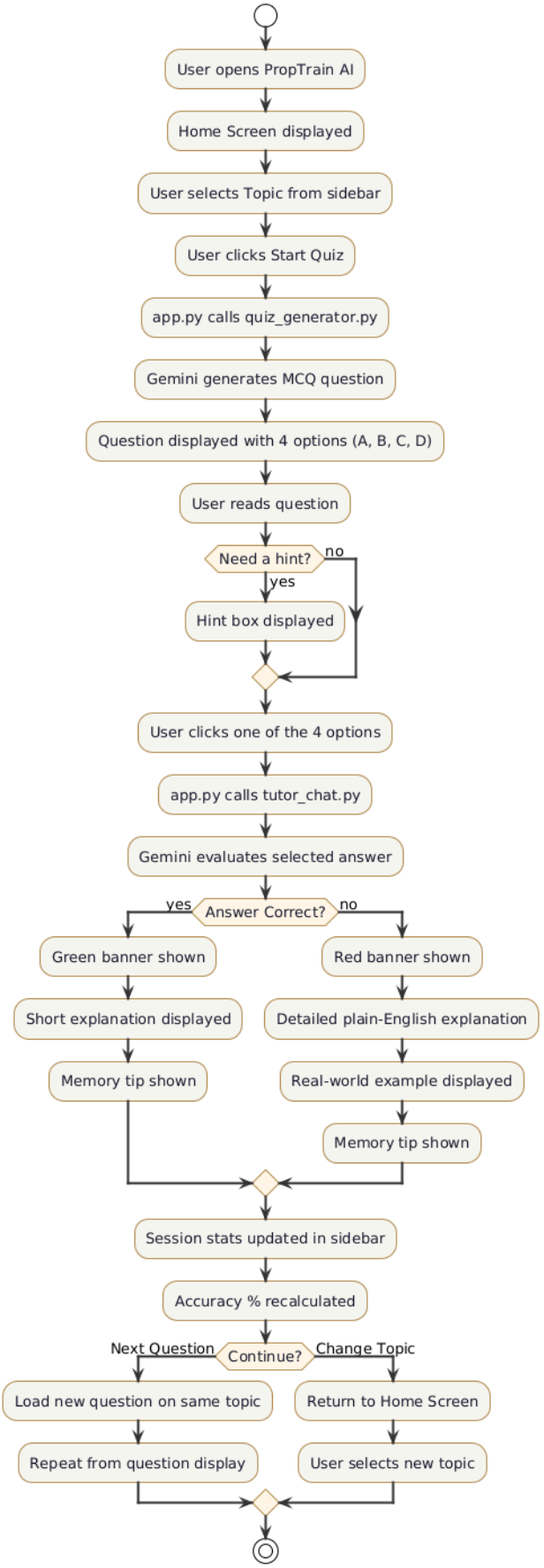
- 10 real estate topics
- Approximately 6000 words of content.

UML Diagrams

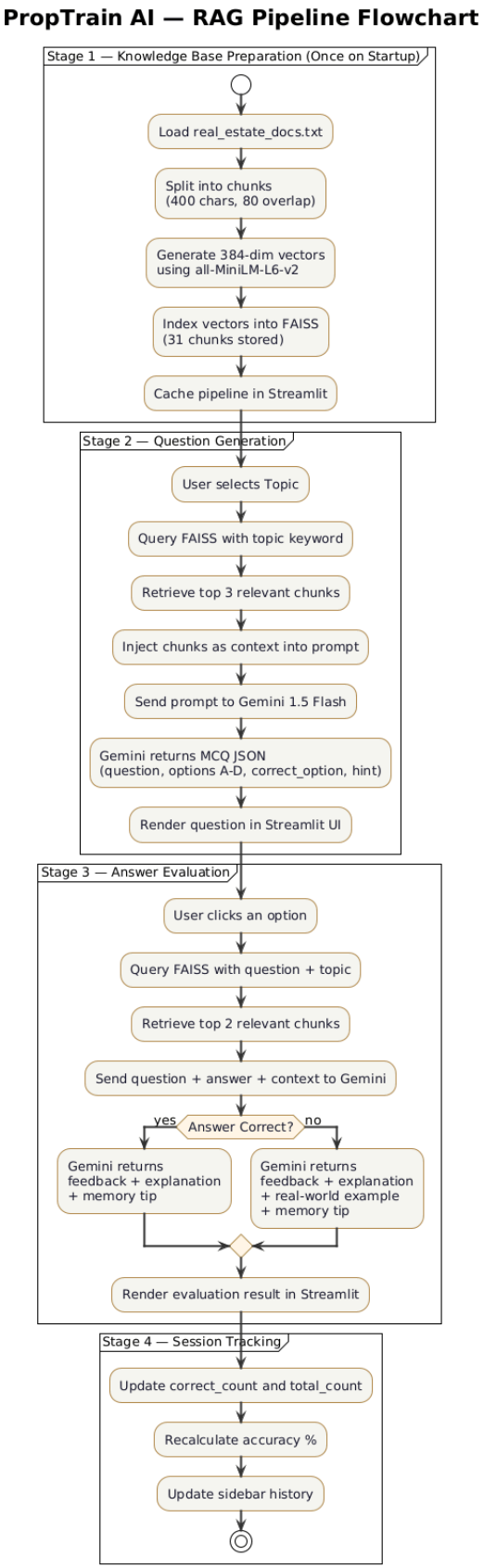
The following UML diagrams have been prepared for this project using PlantUML. All diagram source codes are available separately and diagrams are to be inserted below after rendering

1. Application Flow Diagram.

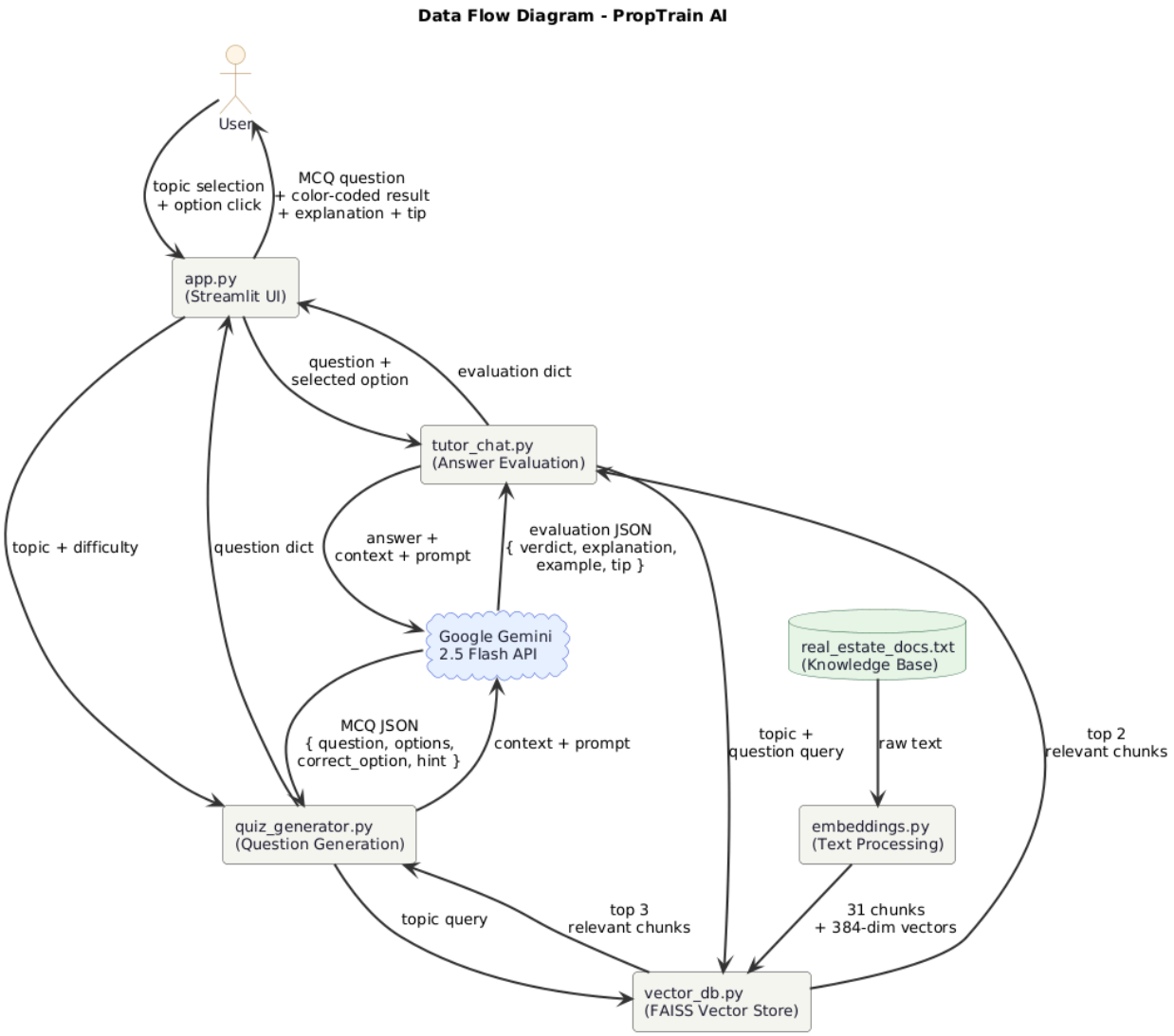
PropTrain AI — Application Flow Diagram



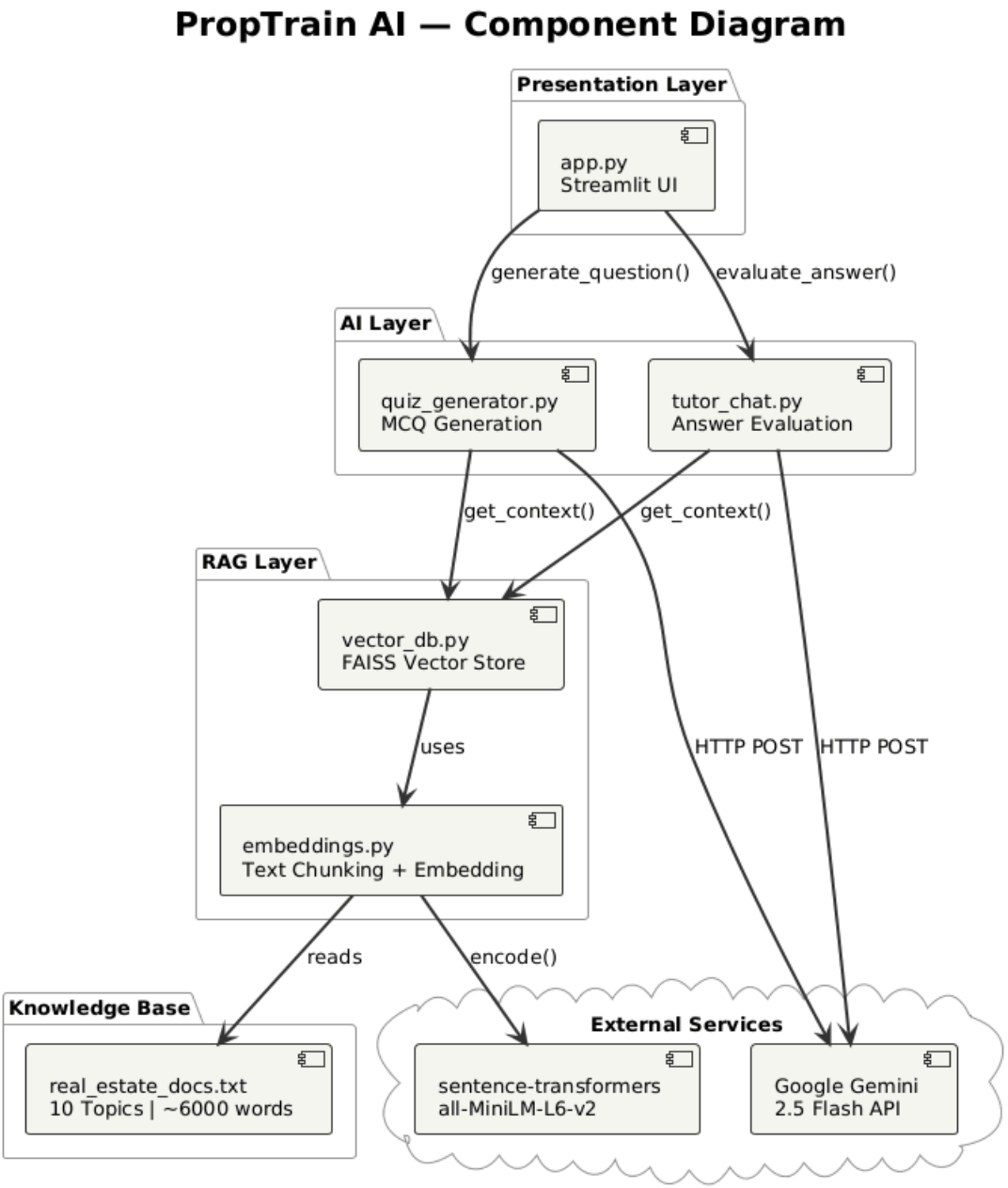
2. RAG Pipeline Flowchart



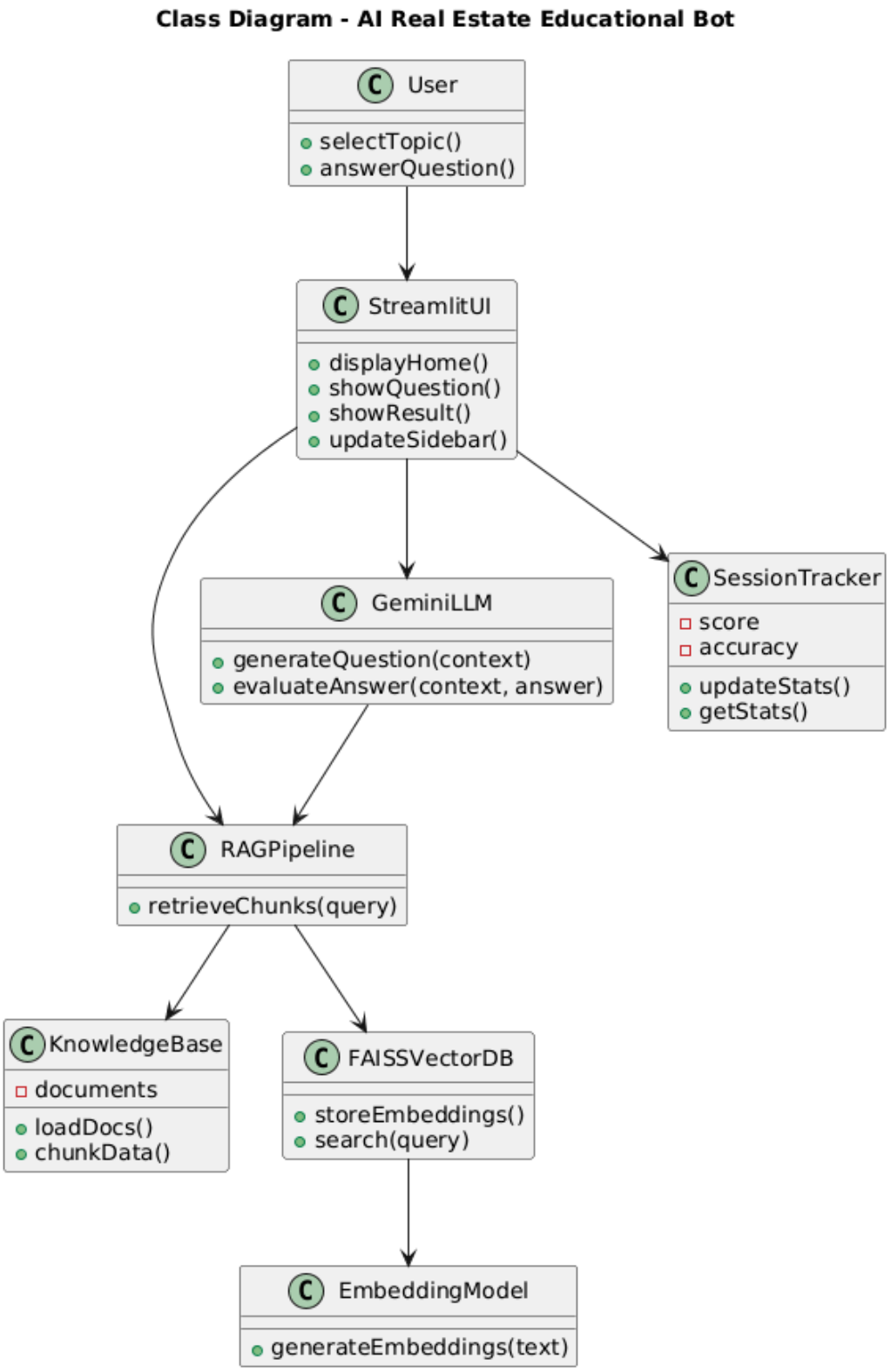
3. Data Flow Diagram



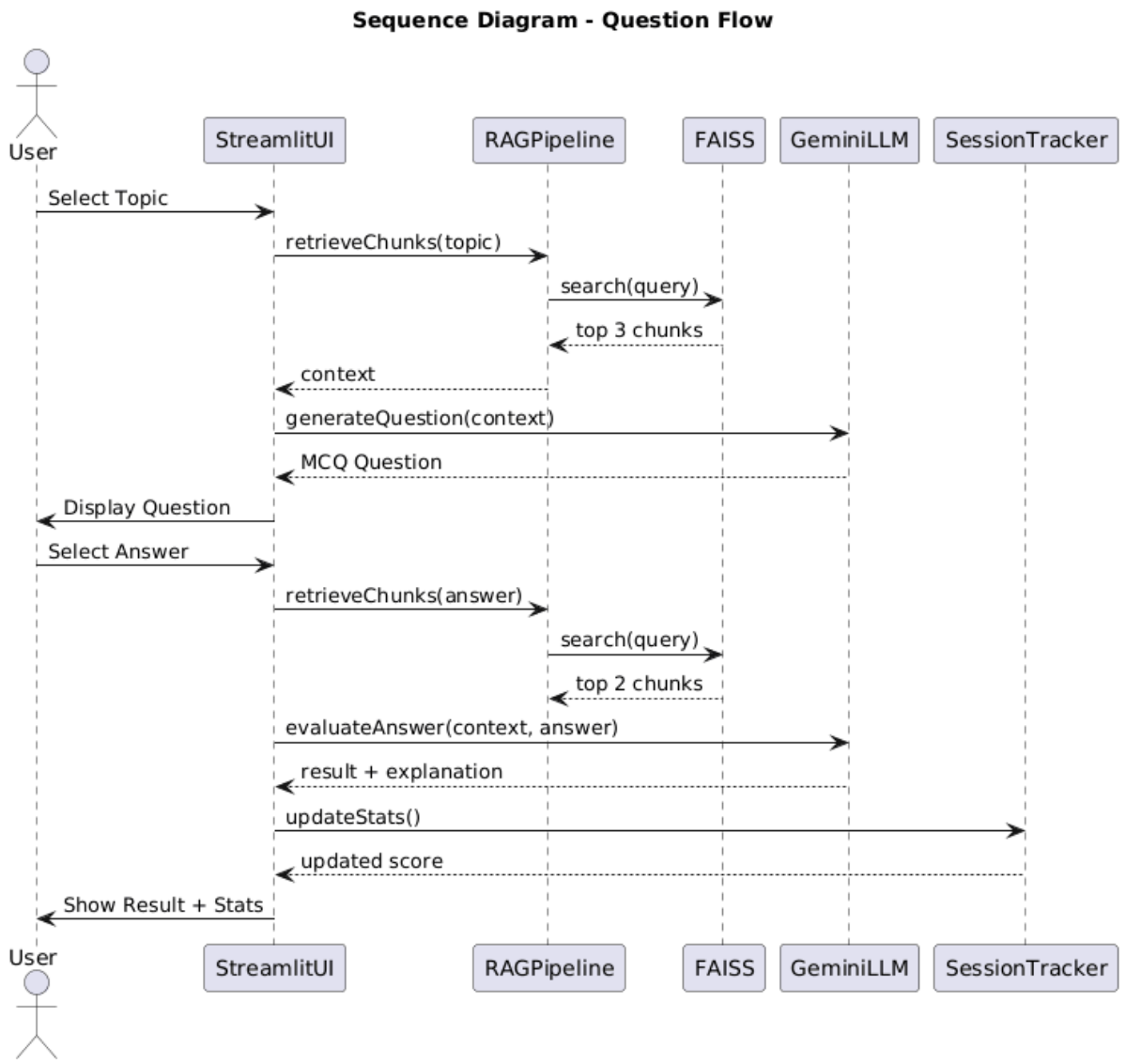
4. Component Diagram



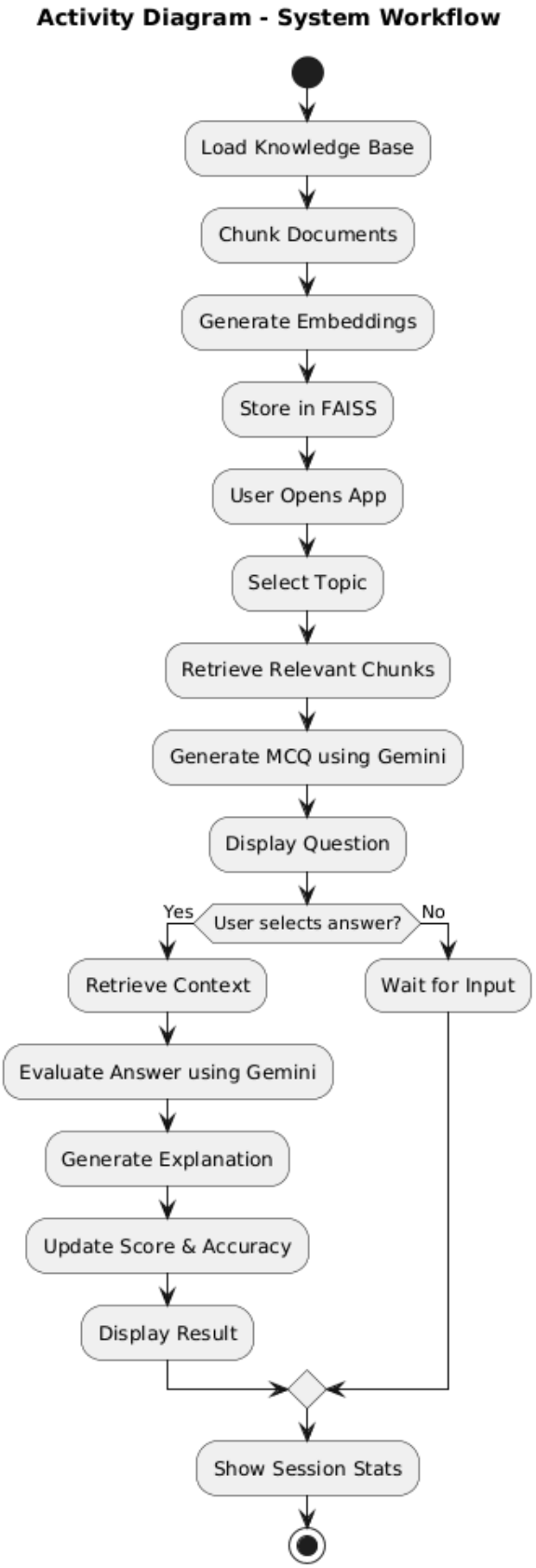
5. Class Diagram



6. Sequence Diagram



7. Activity Diagram



2.3 Tools & Technologies Used

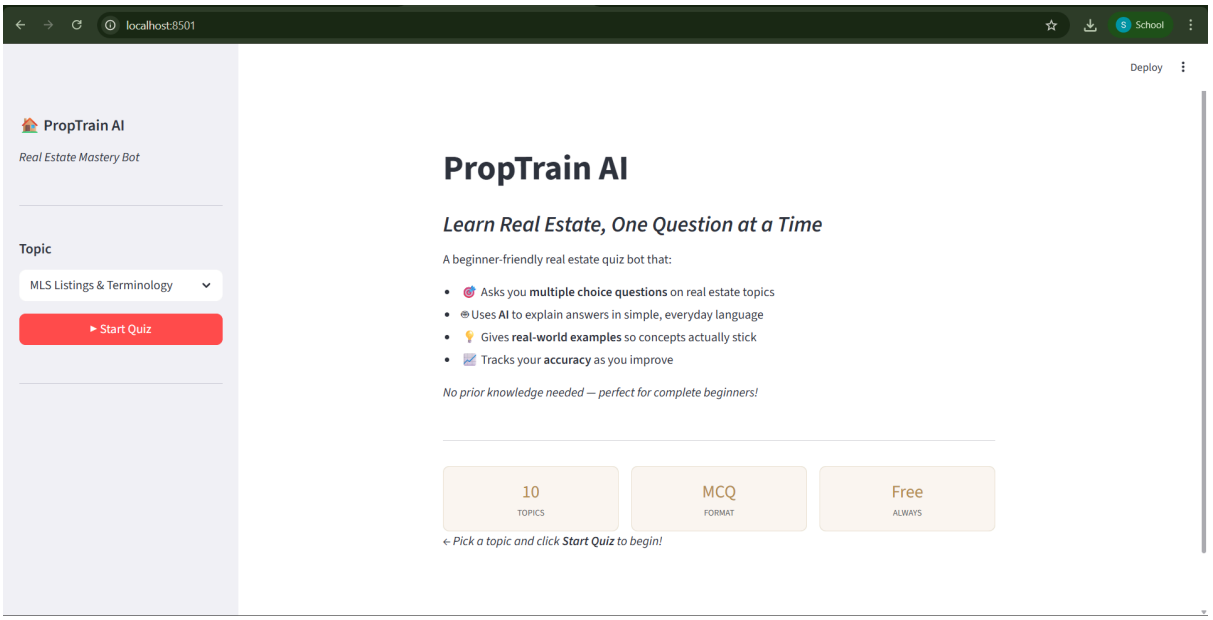
Category	Tool / Technology	Purpose
Programming Language	Python	Used to build the application logic.
UI Framework	Streamlit	Creates the web interface for the quiz.
LLM	Google Gemini 2.5 Flash	Generates MCQ questions and evaluates answers.
Vector Database	FAISS	Stores embeddings and retrieves relevant data.
Embedding Model	Sentence Transformers (MiniLM)	Converts text into vector embeddings.
RAG Pipeline	Custom Python implementation	Connects the application with the Gemini AI model.
API Integration	Gemini API	Used to generate MCQ questions and evaluate user answers using the AI model.
Development Environment	Python	Used to develop the application and implement the RAG pipeline and quiz logic.

3. Results & Output

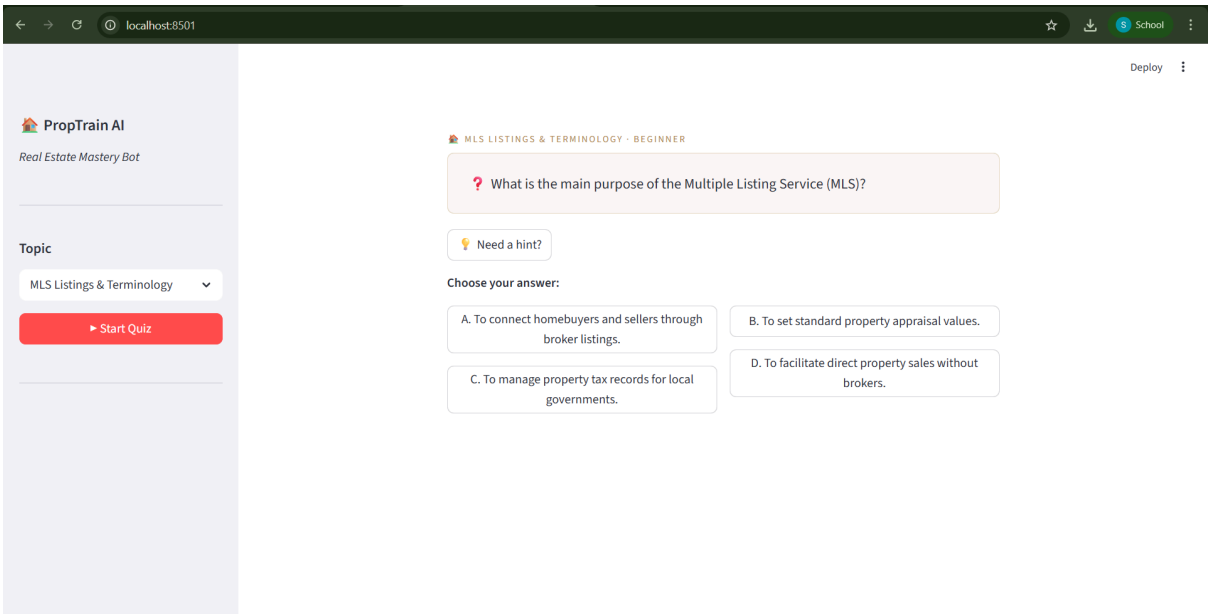
The PropTrain AI system successfully generates quiz questions based on real estate topics and evaluates user answers.

3.1 Screenshots / Outputs

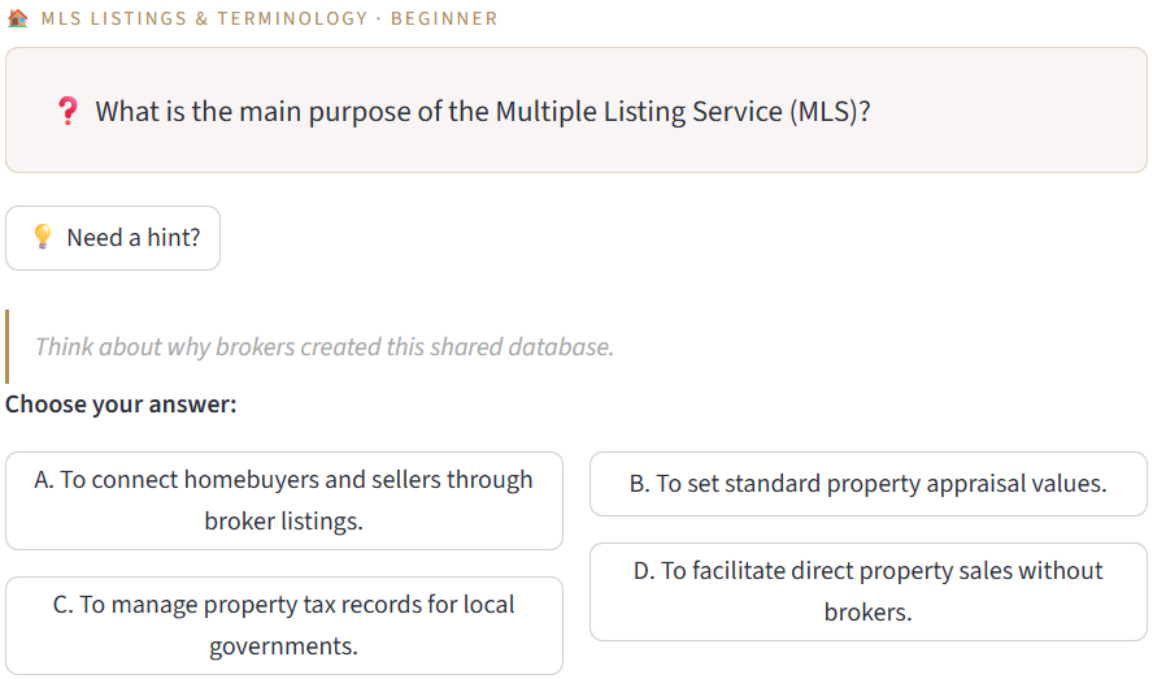
I. Home Screen: PropTrain AI — Home Page



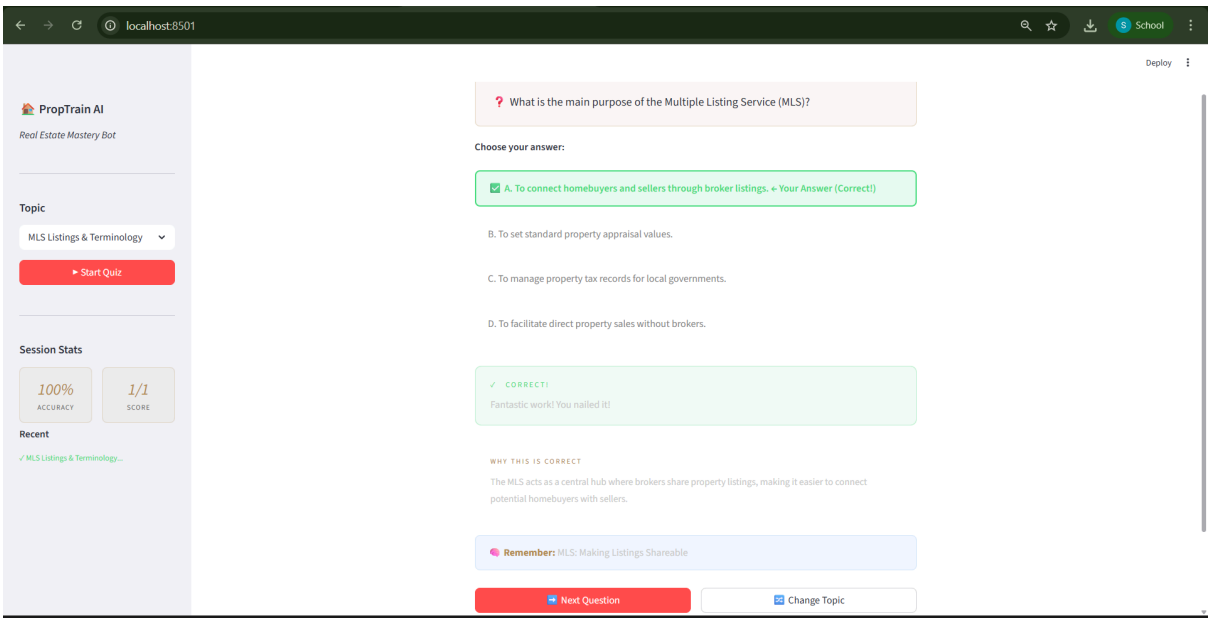
II. Quiz Screen with 4 options: MCQ Question with Options



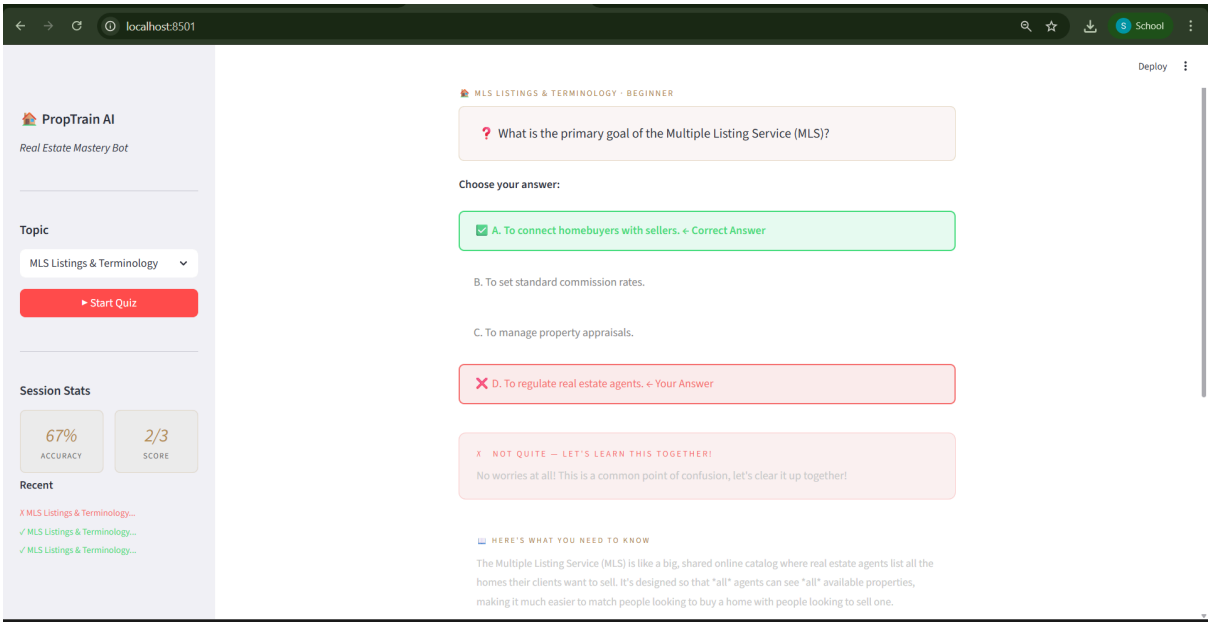
III. Hint visible: Hint System

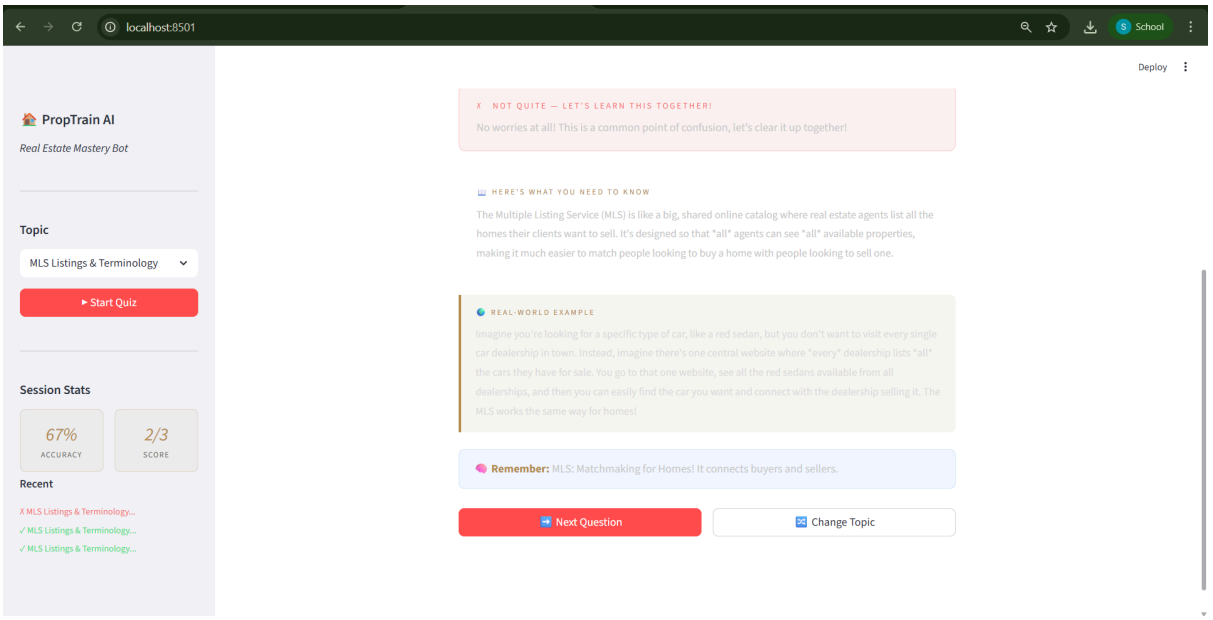


IV. Correct Answer Result: Correct Answer with Explanation

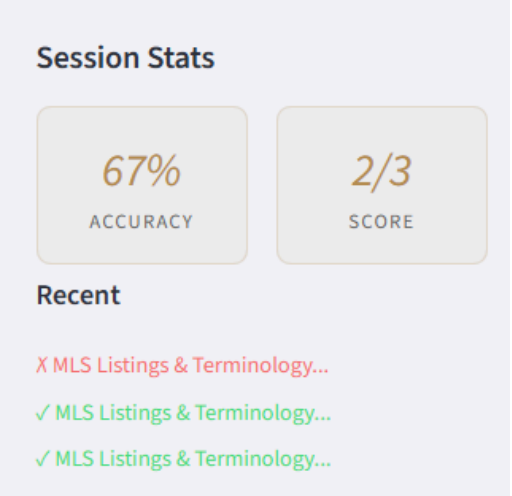


V. Incorrect Answer Result: Incorrect Answer with Real-World Example

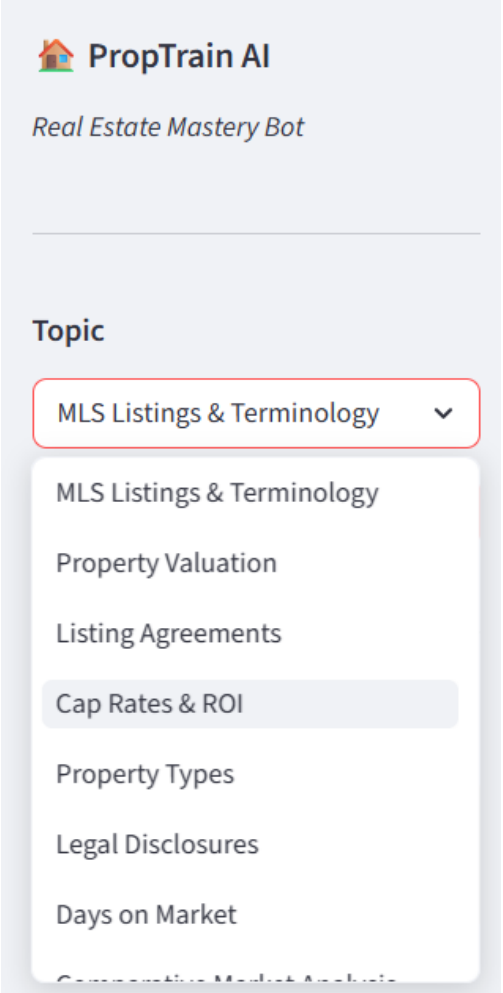




VI. Session Stats in Sidebar: Live Session Accuracy Tracking



VII. All Topics Dropdown: 10 Topic Areas



3.2 Reports / Dashboards / Models

The system generates structured JSON outputs from the Gemini API that include:

MCQ Object:

- Question
- Four options
- Correct answer
- Hint

Evaluation Object:

- Correct/Incorrect verdict
- Explanation
- Real-world example
- Memory tip

3.3 Key Outcomes

- Interactive learning experience for beginners
- Accurate responses using RAG architecture
- Fast retrieval using FAISS vector search
- Clear explanations generated by AI
- Simple and user-friendly interface

The system achieves **1–2 second response time** for both question generation and answer evaluation.

4. Conclusion

PropTrain AI successfully demonstrates how Generative AI can be applied to create an intelligent educational system. By combining Retrieval-Augmented Generation with the Google Gemini model, the system generates meaningful quiz questions and explanations based on real educational content.

The project provides an engaging way for users to learn complex real estate concepts through interactive quizzes. The modular architecture and use of modern AI technologies make the system efficient, scalable, and easy to maintain.

Through this project, the team gained practical experience in **Generative AI, RAG pipelines, vector databases, and AI-based educational applications.**

5. Future Scope & Enhancements

Several improvements can be implemented in future versions of the system:

1. Add **multi-user support with authentication**.
2. Deploy the application on cloud platforms.
3. Integrate a **database for storing user progress**.
4. Expand the knowledge base to include more real estate topics.
5. Add **adaptive difficulty levels** for questions.
6. Implement **voice-based interaction** for accessibility.
7. Replace FAISS with cloud vector databases such as **Pinecone or Weaviate**.
8. Create a **mobile version of the application**.